

CIPHERVAULT

Sistem Brankas Digital untuk Pengamanan Data Menggunakan Kriptografi AES-256 Berbasis Web

1. Latar Belakang

Perkembangan teknologi digital yang pesat membawa dampak signifikan terhadap cara penyimpanan dan pertukaran informasi sensitif. Individu maupun organisasi semakin bergantung pada layanan penyimpanan berbasis *cloud* untuk menyimpan dokumen penting, catatan pribadi, dan berkas rahasia. Namun, mayoritas layanan *cloud* konvensional menyimpan data pengguna dalam kondisi yang dapat diakses oleh penyedia layanan itu sendiri, sehingga menimbulkan risiko kebocoran data baik melalui serangan siber maupun penyalahgunaan dari dalam (*insider threat*).

Statistik global menunjukkan bahwa pelanggaran data (*data breach*) terus meningkat setiap tahunnya. Model keamanan tradisional yang mengandalkan kepercayaan kepada pihak ketiga (server/penyedia layanan) terbukti rentan. Dibutuhkan paradigma baru yang dikenal sebagai *Zero-Knowledge Architecture*, di mana penyedia layanan secara matematis tidak memiliki kemampuan untuk mengakses data pengguna meskipun mereka menginginkannya.

Berdasarkan permasalahan tersebut, kelompok kami mengembangkan CIPHERVAULT, sebuah aplikasi brankas digital berbasis web yang menerapkan enkripsi sisi klien (*client-side encryption*) secara penuh. Seluruh proses kriptografi terjadi di *browser* pengguna sebelum data dikirimkan ke server, sehingga server hanya menyimpan data acak (*ciphertext*) yang tidak bermakna tanpa kunci dari pengguna.

2. Tujuan

Proyek CIPHERVAULT dikembangkan dengan tujuan sebagai berikut:

- Mengimplementasikan sistem penyimpanan *file* dan catatan terenkripsi berbasis web dengan arsitektur *Zero-Knowledge* menggunakan algoritma AES-256-GCM.
- Menerapkan *key derivation function* PBKDF2 dengan iterasi tinggi (310.000 iterasi) untuk memproteksi kunci enkripsi dari serangan *brute-force*.
- Membangun antarmuka pengguna yang intuitif dan aman menggunakan Web *Crypto* API standar *browser* tanpa ketergantungan *library* kriptografi pihak ketiga.
- Menyediakan fitur audit *trail* (*activity logging*) untuk pemantauan aktivitas sensitif pengguna sebagai lapisan keamanan tambahan.
- Mendemonstrasikan penerapan nyata konsep kriptografi modern dalam konteks rekayasa perangkat lunak berbasis web (Laravel + Vite/Tailwind CSS).

3. Tinjauan Pustaka

3.1 AES-256-GCM (Advanced *Encryption* Standard – Galois/Counter Mode)

AES adalah algoritma enkripsi simetris yang diadopsi oleh NIST (*National Institute of Standards and Technology*) sebagai standar enkripsi federal Amerika Serikat sejak tahun 2001. Dengan panjang kunci 256-bit, AES-256 menawarkan tingkat keamanan tertinggi dalam keluarga AES dan digunakan secara luas oleh lembaga pemerintah, militer, dan industri keuangan di seluruh dunia.

Mode operasi GCM (Galois/Counter Mode) merupakan mode *authenticated encryption* yang memberikan dua jaminan sekaligus: kerahasiaan (*confidentiality*) melalui enkripsi CTR dan integritas data (*data integrity/authenticity*) melalui *Galois Message Authentication Code* (GMAC). Properti ini memastikan bahwa *ciphertext* yang telah dimodifikasi oleh pihak tidak berwenang akan gagal saat proses dekripsi, sehingga melindungi sistem dari serangan tampering.

Dalam implementasi CIPHERVAULT, AES-256-GCM dieksekusi melalui Web *Crypto* API (*window.Crypto.subtle*) yang merupakan implementasi kriptografi *native* dan terpercaya yang tersedia di seluruh *browser* modern.

3.2 PBKDF2 (Password-Based *Key Derivation Function* 2)

PBKDF2 adalah fungsi penurunan kunci berbasis kata sandi yang didefinisikan dalam RFC 2898 dan direkomendasikan oleh NIST SP 800-132. Fungsi ini dirancang untuk mengatasi kelemahan mendasar dari kata sandi manusia yang cenderung singkat dan mudah ditebak, dengan mengubah kata sandi tersebut menjadi kunci kriptografi yang kuat melalui proses iteratif.

Mekanisme PBKDF2 menerapkan fungsi pseudorandom (dalam proyek ini menggunakan HMAC-SHA-256) secara berulang sebanyak jumlah iterasi yang ditentukan terhadap kombinasi kata sandi dan salt acak. Penggunaan salt mencegah serangan *precomputation* (*rainbow table*), sementara jumlah iterasi yang tinggi memperlambat laju percobaan *brute-force* secara signifikan. CIPHERVAULT menggunakan 310.000 iterasi, sesuai dengan rekomendasi terbaru OWASP untuk PBKDF2-SHA256.

3.3 *Initialization Vector* (IV) dan *Salt*

Initialization Vector (IV) adalah nilai acak yang digunakan sebagai input tambahan dalam algoritma enkripsi blok mode operasi seperti GCM. IV memastikan bahwa enkripsi data yang identik dengan kunci yang sama akan menghasilkan *ciphertext* yang berbeda-beda, sehingga mencegah analisis pola (*pattern analysis*). Standar untuk AES-GCM merekomendasikan IV sepanjang 96-bit (12 byte).

Salt adalah nilai acak yang digunakan dalam proses *key derivation* untuk memastikan keunikan kunci yang dihasilkan meskipun kata sandi yang digunakan sama. CIPHERVAULT meng-generate salt baru (256-bit/32 byte) untuk setiap operasi enkripsi menggunakan `window.Crypto.getRandomValues()` yang merupakan generator angka acak yang aman secara kriptografi (CSPRNG).

3.4 Arsitektur *Zero-Knowledge*

Zero-Knowledge Architecture (ZKA) dalam konteks penyimpanan data merupakan konsep keamanan modern yang dirancang agar penyedia layanan tidak memiliki kemampuan untuk melihat, membaca, maupun mengakses isi data pengguna. Pada arsitektur ini, seluruh proses enkripsi dilakukan langsung di perangkat pengguna sebelum data dikirim ke server. Dengan demikian, server hanya menerima data dalam bentuk terenkripsi (*ciphertext*), sedangkan kunci dekripsi tetap berada di sisi pengguna dan tidak pernah dikirim ataupun disimpan oleh penyedia layanan.

Konsep ini berbeda dengan sistem keamanan biasa yang hanya menggunakan enkripsi transport seperti HTTPS. HTTPS memang melindungi data saat proses pengiriman melalui internet, namun setelah data sampai di server, data tersebut masih dapat dibaca oleh penyedia layanan. Sementara itu, pada *Zero-Knowledge Architecture*, data tetap terenkripsi bahkan setelah disimpan di server sehingga pihak penyedia layanan tidak mengetahui isi asli data pengguna.

Model keamanan ini memiliki implikasi yang sangat penting terhadap perlindungan privasi dan keamanan informasi. Apabila terjadi kebocoran *database*, peretasan server, ataupun pencurian data, *file* yang tersimpan tetap tidak dapat dibaca tanpa kunci dekripsi yang dimiliki pengguna. Bahkan dalam kondisi tertentu ketika penyedia layanan diminta menyerahkan data kepada pihak lain, data yang diberikan tetap berada dalam bentuk terenkripsi dan tidak dapat dipahami tanpa akses terhadap kunci pengguna.

Pendekatan *Zero-Knowledge Architecture* banyak digunakan pada layanan keamanan modern seperti ProtonMail untuk email terenkripsi, Tresorit untuk penyimpanan cloud aman, serta Signal untuk komunikasi terenkripsi *end-to-end*. Penggunaan konsep ini menunjukkan bahwa keamanan data tidak hanya bergantung pada perlindungan server, tetapi juga pada kontrol penuh pengguna terhadap kunci enkripsi dan akses terhadap data mereka sendiri.

4. Perancangan Sistem

4.1 Arsitektur Sistem

CIPHERVAULT dirancang menggunakan arsitektur dua lapis yang memisahkan tanggung jawab kriptografi dan penyimpanan data secara tegas:

Komponen	Frontend (<i>Browser</i>)	Backend (Server)
Teknologi	Blade, Tailwind CSS, Vite, Web <i>Crypto</i> API	Laravel 12, PHP 8.2, Eloquent ORM, SQLite
Tanggung Jawab	Enkripsi & Dekripsi data, antarmuka pengguna, pengelolaan kunci	Autentikasi pengguna, penyimpanan <i>ciphertext</i> , <i>routing</i> , <i>activity logging</i>
Akses Plaintext	Ya (hanya dalam memori <i>browser</i> , tidak pernah disimpan)	Tidak pernah (hanya menerima dan menyimpan <i>ciphertext</i>)

4.2 Skema Basis Data

Sistem menggunakan empat tabel utama dengan relasi sebagai berikut:

Tabel	Kolom Kunci	Fungsi
users	id, name, email, password	Menyimpan akun pengguna (password di-hash Laravel)
encrypted_files	user_id, original_name, <i>ciphertext</i> , iv, salt	Menyimpan <i>file</i> terenkripsi beserta metadata kriptografi
secure_notes	user_id, title, <i>ciphertext</i> , iv, salt	Menyimpan catatan aman yang terenkripsi
activity_logs	user_id, action, ip_address, user_agent	Audit <i>trail</i> setiap aktivitas enkripsi/dekripsi/hapus

4.3 Alur Proses Enkripsi dan Dekripsi

Berikut adalah alur proses utama kriptografi yang terjadi di sisi klien (*browser*):

Alur Enkripsi:

- Pengguna memilih *file*/menulis catatan dan memasukkan kata sandi enkripsi.

- Salt 256-bit dan IV 96-bit di-generate secara acak menggunakan CSPRNG *browser*.
- Kata sandi + salt diproses oleh PBKDF2 (310.000 iterasi, HMAC-SHA-256) menghasilkan *CryptoKey* AES-256.
- Data dienkripsi dengan AES-256-GCM menggunakan kunci dan IV tersebut.
- *Ciphertext*, IV, dan salt dikonversi ke Base64 dan dikirim ke server melalui HTTPS.
- Server menyimpan *ciphertext*, IV, dan salt ke *database*. Kata sandi tidak pernah meninggalkan *browser*.

Alur Dekripsi:

- Pengguna memilih *file*/catatan untuk dibuka dan memasukkan kata sandi.
- *Browser* meminta *ciphertext*, IV, dan salt dari server melalui API endpoint.
- Kunci diturunkan ulang menggunakan PBKDF2 dengan salt yang diterima dari server.
- AES-256-GCM mendekripsi data sekaligus memverifikasi integritas (GCM authentication tag).
- Jika kata sandi benar, *file* disediakan untuk diunduh/ditampilkan langsung di *browser*. Jika salah, dekripsi gagal dengan pesan error.